Data Systems Project

AY26S1 Prof. Melanie HERSCHEL

Contents

Project Guide 2
Learning objectives 2
Required tools 2
Context 2
Datasets 2
Task 1: Conceptual ER Modeling, Logical Schema Design, and DB implementation 3
1.1 ER Model 3
1.2 Logical Schema Design and Implementation 3
Reporting for Task 1 3
Task 2: Data Acquisition and Population 4
2.1 Data Acquisition 4
2.2. Database Population 4
Reporting for Task 2 4
Task 3: SQL Queries and Initial Performance Results 5
3.1 SQL queries 5
3.1 Performance Measurements 5
Reporting for Task 3 5
Task 4: Indices and their Effect on Runtime 6
4.1 Index creation 6
4.2 Effect on Runtime 6
Reporting for Task 4 6
Appendix A: Required Queries 7
Appendix B: Assessment and Grading Rubric 8
Appendix C: Project logistics 9
Forming teams 9
Working on the project 9
Submissions 10
Viva 10

Project Guide

Learning objectives

- Apply main steps of developing an application based on a relational database system (schema design, data acquisition, data transformation, querying, and indexing) for an example application.
- Gain first-hand experience with databases containing a reasonable amount of data (and to have a taste of writing efficient SQL queries for a real-life database)

Required tools

- PostreSQL
- MS Excel or comparable
- Programming language of your choice
- If you would like to use alternative or additional tools, please seek approval upfront from course coordinator.

Context

Public housing and hawker centres represent two pillars of daily life and social infrastructure in Singapore. Understanding the relationship between HDB resale prices, property attributes (e.g., remaining lease, floor area), and access to affordable dining options (e.g., hawker centres) can support urban planning, housing market analysis, etc.

Using raw datasets from data.gov.sg, your team is tasked with building the data backend to capture and analyse HDB resale data linked with information on nearby hawker centres.

Datasets

- HDB resale data: https://data.gov.sg/collections/189/view
- Hawker centre data: https://data.gov.sg/datasets/d_68a42f09f350881996d83f9cd73ab02f/view?dataExplorerPage=11
- Geolocation data (longitude and latitude)
    - For HDBs: https://github.com/BlueSkyLT/siteselect_sg (only use hdb.csv)
    - Hawker centres: See hawker_geo.csv on NTULearn, extracted from Hawker Centres (GEOJSON)

Task 1: Conceptual ER Modeling, Logical Schema Design, and DB implementation

1.1 ER Model

Design the **conceptual ER model** to support the queries given in Appendix A using the datasets mentioned above. To help you get started, take note of the following.

Central entities that should appear in your ER diagram are:

- Town,
- HDB Flat,
- Resale Transaction
- Hawker Center

To link HDB flats with nearby hawker centres, you should **define three distinct levels of proximity**. You may define these levels of proximity as you like (e.g., based on distance, walking path length, accessibility by public transport). The datasets provided primarily support a distance-based definition, other options may require additional data or API calls. You will need to **model the chosen proximity in your ER** model as well.

Make sure to clearly mark **keys** and **cardinalities** using the (min,max)-notation.

Note that the data contains **more entity types/attributes than necessary** to support the queries of Appendix A. You can decide to add additional attributes to your ER to possibly explore additional aspects you are interested in.

1.2 Logical Schema Design and Implementation

Map the ER diagram to a relational schema in **Third Normal Form (3NF)** or **Boyce-Codd Normal Form (BCNF)**.

Create a database in PostgreSQL and **create the designed schema in SQL**. Take care in defining constraints as precisely as possible to avoid invalid states of the database (constraints include keys, foreign keys, data types, etc.).

Reporting for Task 1

In your report, for Task 1, you have to include:

- A description of your proximity definition
- The complete ER diagram
- All commands to create the schema
- Supplementary material: .sql file with executable commands to create the database

Task 2: Data Acquisition and Population

2.1 Data Acquisition

Starting with **one single file of your choice from the HDB resale dataset** (we recommend not to use the largest one though) and the **remaining datasets**, write code using the programming language of your choice (recommended: Python) for automated and reproducible **parsing, extraction, and transformation** of the data needed to populate your database.

Document which transformations to the original data are required to fit your (well designed ;-)) database. Note that you may need to perform additional computations, use external services, or supplementary datasets to **acquire data for the proximity levels** between HDBs and hawker centres.

2.2. Database Population

Your code should **produce a .csv file for each table** of your database, such that these can be imported using **bulk loading/import** capabilities of the data system.

It is expected that **you will run into errors and inefficiencies** when trying to import the data from the csv files into the database. Typical examples include data types in the csv not matching those declared in the database, some parsing errors resulting in empty values, duplicated keys or data not aligned with foreign key constraints, etc. You are likely to get error messages, which are to be resolved by understanding what went wrong and **fixing your code in an iterative way**. **Document the (unexpected) errors or issues encountered during data loading.**

Reporting for Task 2

In your report, for Task 2, you have to include:

- A description of required transformations and computations that you implemented in your program.
- A reflection on errors and other issues encountered during data loading
- Supplementary material: executable code written to generate csv files

Task 3: SQL Queries and Initial Performance Results

3.1 SQL queries

Write **SQL queries** to answer the queries **Q1 – Q5 in Appendix A**.

Define an **additional query Q6** of your choice that requires joining all tables of your database. Write the corresponding SQL query.

Execute each query on the database you populated in the previous tasks. Check the result to make sure it matches the requirements. You may need to refine your queries to match the requirements. We explicitly encourage you to try and test your queries to debug them.

For each query, look at the result data and **report at least one insight you can gain from the query result**. Note that an insight can also be the absence of an expected effect, so statements such as "From this query result alone, we cannot conclude that there is a significant effect of X on Y" are also valid insights.

3.1 Performance Measurements

For your initial instance with data from one HDB resale file loaded so far, run each query **5 times**, record each query running time, and **compute the average running time**.

Gradually increase the database size and perform runtime measurements for each increment. To do so, add one by one another of the remaining 4 HDB resale data files to the database, reusing the same code to extract, transform, and load the data. If this causes (new) errors, fix your data acquisition code. Note that this may also require emptying the database and repopulating it.

Keep track of the database size and the runtime for all six queries as you gradually increase the database size. In the end, you will have 5 measurements to be averaged for 6 queries on 5 different database sizes.

Use Excel or other tool to **plot database size vs runtime for each query**. **Interpret** the performance data you have collected. How does database size affect runtime of the different queries?

Reporting for Task 3

In your report, for Task 3, you have to include:

- The SQL queries you wrote
- Screenshots of the query results
- Figures plotting your performance measurements and discussion of results
- Supplementary material: .sql file with the executable SQL queries, raw measurement file (Excel or csv; expecting 150 measurements in total)

Task 4: Indices and their Effect on Runtime

4.1 Index creation

Some of your queries may be slow or may be accelerated using an appropriate index. We recommend using the **EXPLAIN facility** available in pgAdmin to get an idea of the execution plan used for each query. Based on this, try to speed queries up by **creating suited indices using the CREATE INDEX statement**.

4.2 Effect on Runtime

For each index you created, test if it has a noticeable **effect on query runtime**. To do so, run the query on the **largest database** (i.e., the one that contains all transaction data from all HDB files) 5 times using the index and compare with the non-optimized runtime on the same database size obtained as part of Task 3.

Plot the average runtimes for each query with and without indexes in a bar chart using Excel.

Reporting for Task 4

In your report, for Task 4, you have to include:

- The CREATE INDEX statements you used
- The performance plot and discussion of results
- Supplementary material: .sql file with the CREATE INDEX statements, raw measurement file underlying the plot (Excel or .csv).

Appendix A: Required Queries

Q1: Price-per-Square-Meter Across Towns

Write a query to calculate the average price per square meter () for 4-Room and 5-Room flats sold within a 3-year range available in your loaded data. That is, if you opted for the file with prices from the 1990s, then choose a 3-year range of the 90s (e.g., 1995 – 1998).

Q2: Remaining Lease vs. Transaction Price

Categorize sales transactions into remaining lease buckets (e.g., less than 50 years, between 50 – 70 years). You can do so using `CASE`-expressions available in SQL. Then, compute the median resale price, average price per square meter, and total transaction volume for each bucket per town.

Q3: Resale Price for Towns with Large Flats

Identify the top 10 towns with the highest average floor area for 3-Room flats. You can do so using the `LIMIT`-clause in SQL. For these top towns, retrieve the highest resale transaction amount per year for a 3-Room flat. We recommend using subqueries or the `WITH`-statement for this query.

Q4: Hawker Centre Proximity Effect

Find the price differential between HDB flats of closest proximity class and HDB flats in the furthest proximity class you defined. Your output should have the following schema:
(town, flat_type, avg_price_near_hawker, avg_price_far_hawker, price_diff_percentage)
Hint: the price_diff_percentage is computed as
(avg_price_near_hawker – avg_price_far_hawker) * 100 / avg_price_far_hawker

Q5: Town Level Price Near Fresh Market Hawkers

For all towns, compute the average resale price per proximity class (you have defined at least three classes) for flats that are in proximity to a hawker with fresh market stalls. Filter the results to only return towns where the average resale price exceeds a budget limit (in SGD) of your choice, making sure the result is neither empty nor contains all towns.

Appendix B: Assessment and Grading Rubric

Task	Weight	Adequate	Good	High standard
Task 1: Modeling	25%	Missing a key aspect in ER diagram, un-normalized tables, poor choice or definition of constraints	Some details are missing in ER diagram, minor normalization flaws, constraints mostly well-chosen and defined	Clean and complete ER diagram, fully normalized schema, constraints are consistently well-chosen and correctly defined.
Task 2: DB Population	10%	Rudimentary data transformation, incomplete loading due to unhandled errors	Functional script that allows to automatically load most data into the DB, with attribute values mostly matching the schema semantics	Robust, automated, and thoughtful data transformation to faithfully match schema semantics, loading all data correctly into the DB
Task 3: Querying	25%	Executable queries with multiple semantic errors; incorrect query results;	Queries yield correct results but are written in an overly complicated or verbose way.	Correct SQL queries that are written for optimal readability.
Task 4: Indexing	20%	No systematic choice of created indexes; incomplete execution metrics	Mostly reasonable choices of indexes created; complete execution metrics and acceptable discussion of results	Clear justification for each index created; complete execution metrics; detailed and insightful discussion of results.
Deliverable Quality	5%	Poor formatting; poor coding and commenting standards; disorganized code; some missing deliverable items;	Clear and complete documentation; minor gaps in coding standards and commenting;	Professional report formatting; clean code; well-commented code;
Viva	15%	Rudimentary explanation of solution; lacks understanding of design choices or team contributions.	Most questions answered adequately; evidence of general understanding; some difficulty when it comes deeper discussion	Well-articulated and thoughtful responses; clear justification and defence of proposed solution; evidence of mastery of submitted work.

Appendix C: Project logistics

Forming teams

- Form teams during Week 3
- Standard group size 6 to meet as much as possible
    - We expect 21 groups of 6 students, and2 groups of 5 students
    - If the number of teams is not minimal, we will adjust teams by moving some students to other teams of 5.
- Team registration: Data Systems Project - Team members – Fill out form
    - Choose team name
    - List member names using student ID name
    - Indicate availability for daytime viva if possible

Working on the project

- **Equal contributions:** There is one submission per team, but all members of the team must equally contribute to the project. Should issues of equal contribution arise, these will be handled on a case-by-case basis and may result in individual members of a team incurring penalties.
- **AI usage**: We encourage you to use AI in accordance with NTU Policies and Guidelines. You are, however, responsible for all parts of the project submission and should be able to discuss and defend all aspects of the project during the viva. If you used AI, please declare its use in the report. Also feel free to share interesting experiences about what AI was useful / not useful for in hindsight, by adding a short reflection paragraph to your report. We are always curious to see how AI capabilities evolve ;-)
- **Team coordination and exchange:** It is perfectly fine to split the work among team members. To do so, we recommend you coordinate and communicate clearly, not only who does what, but also on the choices made and results obtained. This is to ensure that all team members understand the whole project in detail, which will be important for the viva.
- **Time management:** We recommend you work on the project regularly and continuously through the semester. Tasks 1 and 2 can be completed before recess week. Working early on these task supports load balancing and will support your learning for Quiz 1. Similarly, working on the remaining tasks before Quiz 2 will give you insights that support your learning for Quiz 2.
- **Ambiguous by design:** The project description deliberately leaves some aspects open or ambiguous. This mimics the situation in real work environments where not everything is clearly specified upfront. Within your team, discuss and set assumptions to justify your choices.

Submissions

- The project guide lists all required content and digital artifacts for a complete submission (see boxed frames).
- Submission will be done via NTULearn.
- Submission is due by Friday, end of day, of Week 13.
- Late submissions will be penalized by 5 marks a day. Submissions not received by the day preceding the viva date will receive 0 marks on the viva.

Viva

- Each team will have 15 minutes to briefly present a specific aspect of their solution and answer questions on any part of their solution.
- For these, we will use both the 3-hour time slot on Thursday evening during Week 14, and another 3 hours during daytime. If available during daytime, please indicate so in the team form as we will need to accommodate all students.
- All team members must be present for the viva. Absence without a valid excuse (MC, LOA) will result in 0 marks for the project component of CA for the absentee(s).
- The viva will be on site. There is no option to take it online.